End Goal and Next Steps After the Simulator

Oct 16, 2026 · @Vince

End goal

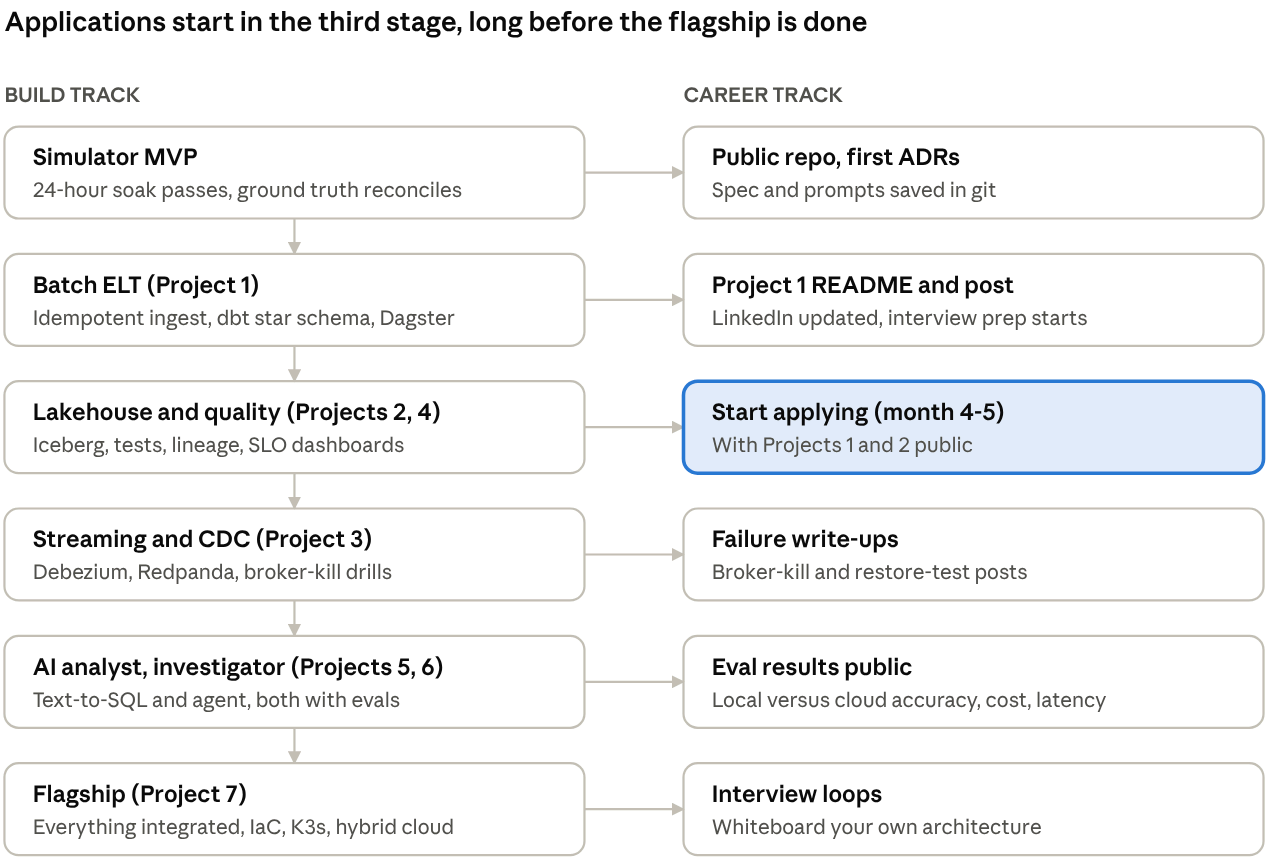
The end goal is a **job as an AI-augmented Data Engineer** (Data Platform, Cloud, or AI Infrastructure track), backed by one flagship project that proves you can design, build, operate, secure, and troubleshoot a modern data and AI platform end to end. The simulator is not the goal; it is the data source that makes the real work possible.

What you will have at the end

- **A flagship platform**, reproducible from one repo: a simulated Vietnamese logistics company feeding CDC and batch pipelines, a lakehouse (bronze, silver, gold), data quality gates, observability, and an eval-backed AI analyst and incident investigator.
- **Operating experience**, not just installation: a homelab with backups you have restored, failures you have injected and recovered from, and measured numbers (latency, throughput, RTO and RPO).
- **Architecture judgement**, written down: ADRs for each major trade-off, which you can defend in an interview.
- **A public portfolio:** GitHub repos with strong READMEs and diagrams, short write-ups, and quantified resume bullets.
- **Interview readiness:** system design practice, SQL fluency, and a story bank, with applications started well before the project is finished.

Why this goal. Employers hire for operated systems and sound trade-offs, not for tool names. A software engineer who can show a reliable, monitored, tested data platform with measured results is a credible data engineering candidate. This is my reasoning from the plan, not a verified hiring statistic.

Path from simulator to job offer



build track and career track · 6 stages, applications from stage 3

Each build stage produces something public that feeds the career track on its right. The shaded step is the one people tend to delay: applying around month 4-5, once the first two projects are published. Stages are drawn in order, not to scale; the roadmap in the plan document puts them across about 10 months.

Right after the simulator MVP: the next 30 days

Once the MVP passes its 24-hour soak test and reconciles cleanly, the first platform project starts. Keep building simulator phases 2-4 (weather, money and planted patterns, faults) alongside it, but do not let them block the platform. Backfill mode arrives in simulator phase 3; until then, the platform works on live data that accumulates.

Week 1: deploy and land data

- [ ] Deploy the `sim` profile to the Ubuntu server and re-run the reconcile script there.

- [ ] Create a separate lake Postgres (never the TMS database) and the MinIO bucket layout: landing, bronze, silver, gold. The ground-truth bucket stays separate with its own credentials.
- [ ] Write an idempotent Python ingestion that lands orders, shipments, and scans into bronze with metadata (`ingested_at`, source, offset).

Week 2: transform

- [ ] Create the dbt project with staging models, uniqueness and not-null tests, and source freshness checks.
- [ ] Add Dagster assets for bronze, silver, and gold with a daily schedule.
- [ ] Dedupe by `event_id` and quarantine rejects in silver instead of dropping them.

Week 3: model and show

- [ ] Build the gold star schema with an explicit grain per table (`fact_shipment`, dimensions, KPIs such as on-time-in-full and depot dwell time).
- [ ] Build the first Grafana dashboard.
- [ ] Reconcile gold against the ground-truth log and record any mismatch as a finding.

Week 4: operate and publish

- [ ] Set up restic backups and do a real restore test onto a clean location.
- [ ] Write the first two ADRs (for example, Dagster versus Airflow, DuckDB versus ClickHouse).
- [ ] Publish Project 1's README with an architecture diagram and a measured result.
- [ ] Update LinkedIn to say you are building this publicly.

Platform stages and what each proves

Stage	What you build	What it proves to an employer
Simulator (prerequisite)	Weather-driven, fault-injecting 24/7 source with ground truth	You can design a source system and reason about causality and failure
Batch ELT (Project 1)	Idempotent ingestion, dbt star schema, Dagster orchestration, tests	Core data engineering: modelling, reliability, reruns and backfills

Stage	What you build	What it proves to an employer
Lakehouse (Project 2)	MinIO, Parquet, Iceberg, DuckDB; schema evolution, time travel, compaction	Storage and format trade-offs
Quality and observability (Project 4)	Contracts, Soda or dbt tests, lineage, Prometheus and Grafana SLO dashboards, alerting	You operate systems, not only build them
Streaming and CDC (Project 3)	Debezium, Redpanda, consumers, ClickHouse; late and duplicate events, broker-kill drills	Distributed-systems thinking and failure handling
AI analyst (Project 5)	Text-to-SQL with RAG over dbt docs, read-only role, 50-question eval	You integrate LLMs reliably, with measurement
AI investigator (Project 6)	Read-only agent that diagnoses failures, scored on 20 injected faults	You build agents with guardrails and honest evaluation
Flagship (Project 7)	Everything integrated, IaC, single-node K3s, hybrid cloud, cost and power report, full ADR set	End-to-end ownership and architecture skill

The order is deliberate: batch before streaming, a solid gold layer before any AI, and measurement at every step. The roadmap puts these across about 10 months at 10-12 hours a week.

Definition of done

The goal is reached when these are true and written down with measured numbers. Targets marked (plan) are the example targets from the plan, not guarantees; set your own after benchmarking.

- [ ] The platform runs continuously for 30 days with monitoring, and alerts fire correctly on injected faults.

- [ ] A rerun and a backfill produce no duplicates, verified by reconciling gold against the ground-truth log.
- [ ] The platform rediscovers all eight planted patterns from the simulator's answer key.
- [ ] GPS event to live map p95 under 5 s, and gold freshness under 15 minutes (plan).
- [ ] A full restore onto a fresh host succeeds within the stated RTO of 1 hour and RPO of 15 minutes (plan), with the measured times recorded.
- [ ] A broker-kill drill is documented: duplicates, loss, recovery time, and the fix.
- [ ] The text-to-SQL assistant reports accuracy on a 50-question eval, local model versus a cloud model, with latency and cost.
- [ ] The incident investigator is scored on 20 injected faults, with its root-cause accuracy reported honestly.
- [ ] At least six ADRs exist covering the major trade-offs, and the flagship repo rebuilds from one command.
- [ ] Resume, LinkedIn, and the repo READMEs show quantified results, and applications are out.

Turning the work into interviews and offers

Start applying around month 4-5, after Projects 1-2 are published, not at the end. Real interviews are the best feedback, and your software engineering background makes you competitive earlier than you may expect.

1. **Publish as you go.** Each project gets a public repo with a README covering the problem, an architecture diagram, trade-offs, a measured benchmark, and an ADR folder.
2. **Write one short post per project.** The failure experiments (broker kill, schema drift, restore test) make the best material.
3. **Quantify resume bullets** with your own measured numbers. Do not copy example figures.
4. **Target the right roles:** data engineering, data platform, and cloud or infrastructure roles where a software background is an advantage.
5. **Practise the story format:** problem, options considered, decision, what broke, what you would change.
6. **Keep interview prep running:** weekly SQL practice (window functions, joins, optimisation), a system design problem every other week, and a behavioural story bank.

7. **Be ready to whiteboard your own architecture** and defend each trade-off, including what you would do at 100× scale.

Market demand claims in this plan are observed patterns or predictions, not verified statistics. Check current job postings in your target market as you go and adjust the skills list.

Risks and how to avoid them

Risk	What to do
Perfecting the simulator for months	Build it in tiers; only the MVP blocks the platform. Time-box each simulator phase
16 GB RAM runs out	Use Compose profiles, never run core + streaming + ai together, cap Redpanda and ClickHouse, upgrade to 32 GB first
Installing too many tools	One orchestrator, one quality tool, one OLAP engine; skip the optional list until the essentials work
AI before the data is solid	Build agents only after gold is reconciled against ground truth
Presenting assumptions as findings	Weather effect sizes and planted patterns are modelling assumptions; document them in ADRs and never call them empirical
Applying too late	Start at month 4-5 with the first two projects published
Unverified facts in the spec	Verify depot coordinates, Vietnam's address structure, and Open-Meteo terms before relying on them
Running out of steam	Keep the weekly schedule realistic (10-12 hours), keep one rest day, and ship small milestones

Keeping your work

This chat is incognito, so I have no memory of it in future conversations, and I can't confirm how long these documents stay available. Export them now rather than relying on them being here later.

- [ ] Export all four docs (Markdown or PDF, or to Google Docs) and store them in a private git repo.
- [ ] Save the simulator documentation as `docs/SPEC.md` in the `logistics-sim` repo; the Claude Code prompt depends on it.
- [ ] Copy the Claude Code prompt and the follow-up prompts into a file in the repo, for example `docs/prompts.md`.
- [ ] If you start a new chat later, paste or upload `docs/SPEC.md` and this document so the context is restored.